RouteWise

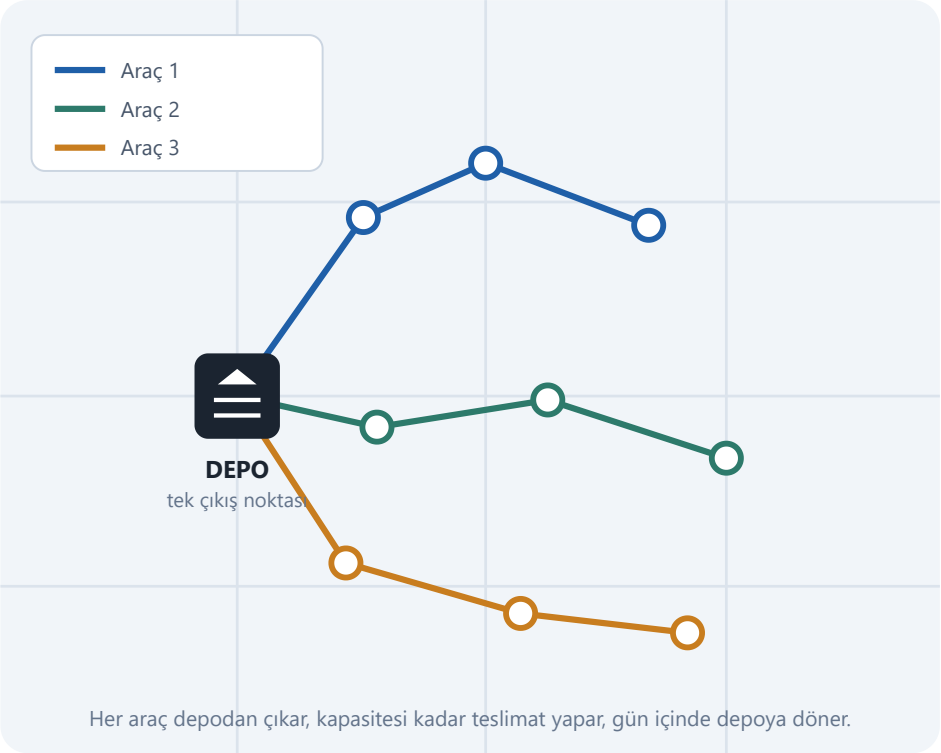
Smart Dynamic Route & Fleet Optimization

Lojistik Dağıtım Süreçlerinde Dinamik Rota ve Araç Optimizasyonu Sağlayan Akıllı Yazılım Sistemi

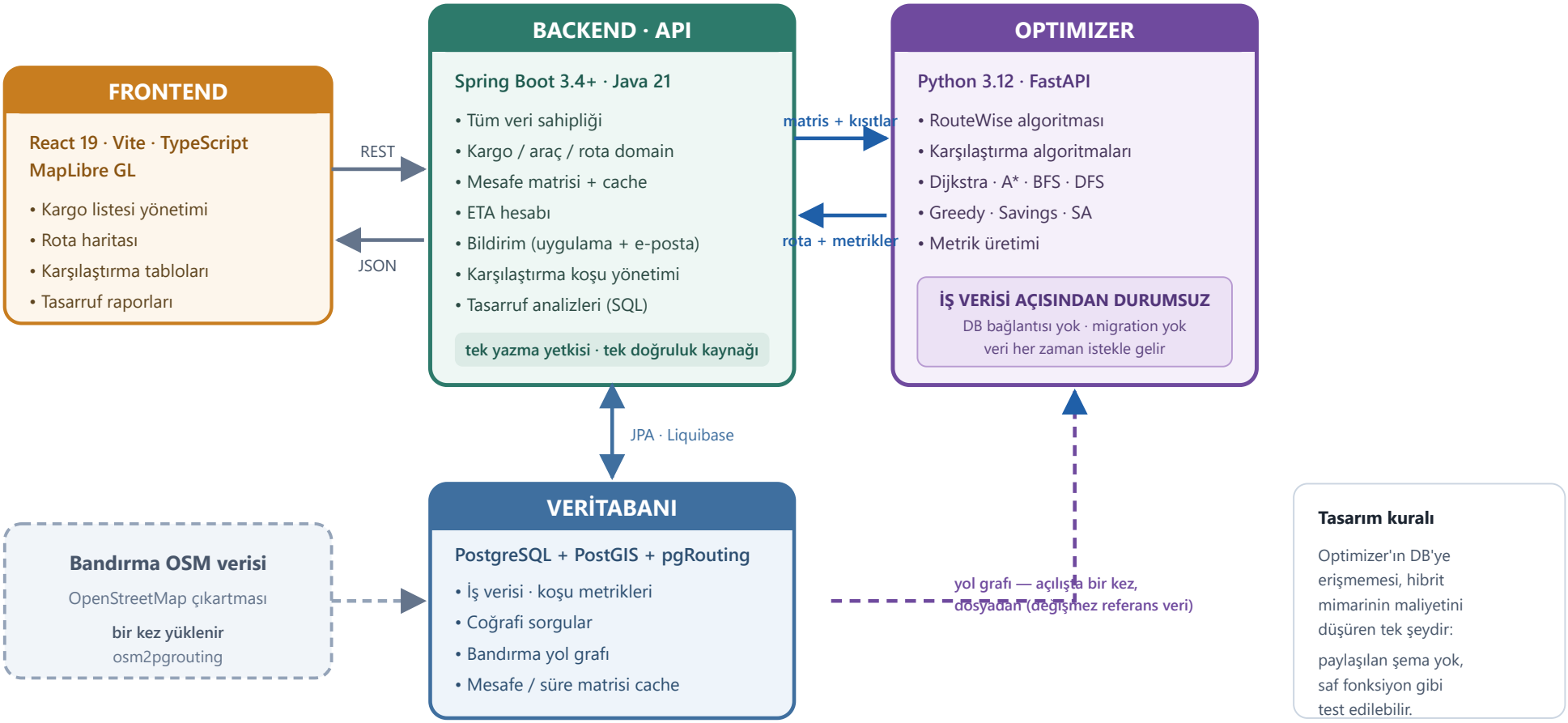
Kapsam: Bandırma içi kargo dağıtımı — tek dağıtım merkezi
Problem: CVRP + maksimum rota süresi
Repo: github.com/AllenVB/RouteWise
Doküman: Sistem mimarisi şeması · 10.10.2026

Sert kısıt: araç kapasitesi | Sert kısıt: çalışma günü süresi

Amaç: mesafe | yakıt | sürücü maliyeti | teslimat önceliği

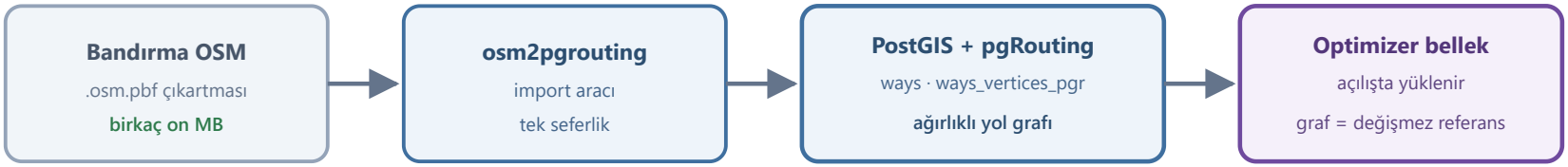


Her araç depodan çıkar, kapasitesi kadar teslimat yapar, gün içinde depoya döner.

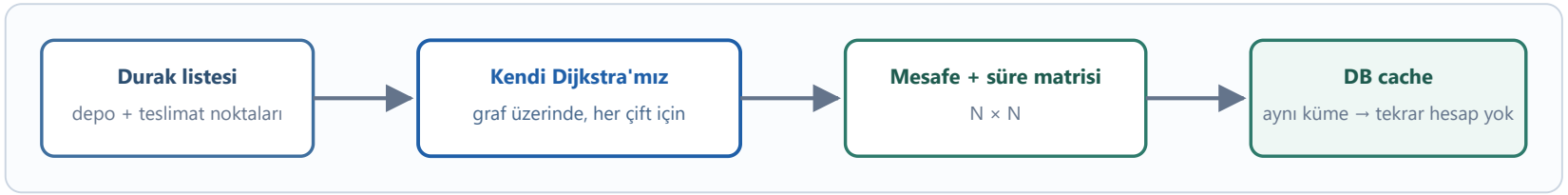

FRONTEND
React 19 · Vite · TypeScript
MapLibre GL
• Kargo listesi yönetimi
• Rota haritası
• Karşılaştırma tabloları
• Tasarruf raporları
REST
JSON
BACKEND · API
Spring Boot 3.4+ · Java 21
• Tüm veri sahipliği
• Kargo / araç / rota domain
• Mesafe matrisi + cache
• ETA hesabı
• Bildirim (uygulama + e-posta)
• Karşılaştırma koşu yönetimi
• Tasarruf analizleri (SQL)
tek yazma yetkisi · tek doğruluk kaynağı
matris + kısıtlar
rota + metrikler
OPTIMIZER
Python 3.12 · FastAPI
• RouteWise algoritması
• Karşılaştırma algoritmaları
• Dijkstra · A* · BFS · DFS
• Greedy · Savings · SA
• Metrik üretimi
İŞ VERİSİ AÇISINDAN DURUMSUZ
DB bağlantısı yok · migration yok
veri her zaman istekle gelir
JPA · Liquibase
VERİTABANI
PostgreSQL + PostGIS + pgRouting
• İş verisi · koşu metrikleri
• Coğrafi sorgular
• Bandırma yol grafı
• Mesafe / süre matrisi cache
Bandırma OSM verisi
OpenStreetMap çıkartması
bir kez yüklenir
osm2pgrouting
yol grafı — açılışta bir kez,
dosyadan (değişmez referans veri)
Tasarım kuralı
Optimizer'ın DB'ye erişmemesi, hibrit mimarinin maliyetini düşüren tek şeydir:
paylaşılan şema yok, saf fonksiyon gibi test edilebilir.

Yol Grafı ve Mesafe Verisi

VERİ HATTI — bir kez kurulur



MATRİS HESABI — koşu başına



Harici rota API'sine ihtiyaç kalmadı

Kapsamın Bandırma'ya daralması bunu mümkün kıldı.

Önceki taslakta (OpenRouteService)	Yol grafıyla
Günlük istek kotası	**sınırsız sorgu**
Matris başına ≈59 konum sınırı	**konum sayısı serbest**
API key yönetimi	**gerekmez**
Demo gününde internet riski	**tamamen yerel**
Ham graf alınamaz	**graf elimizde**

Graf olmadan Dijkstra / A* / BFS / DFS karşılaştırması zaten yapılamazdı.

Süre hesabı

OSM her yol parçasında **maxspeed** taşır.

süre = segment uzunluğu / hız sınırı

hız bilgisi yoksa → yol tipine göre varsayılan

+ durak başına **servis süresi** (teslim/indirme)

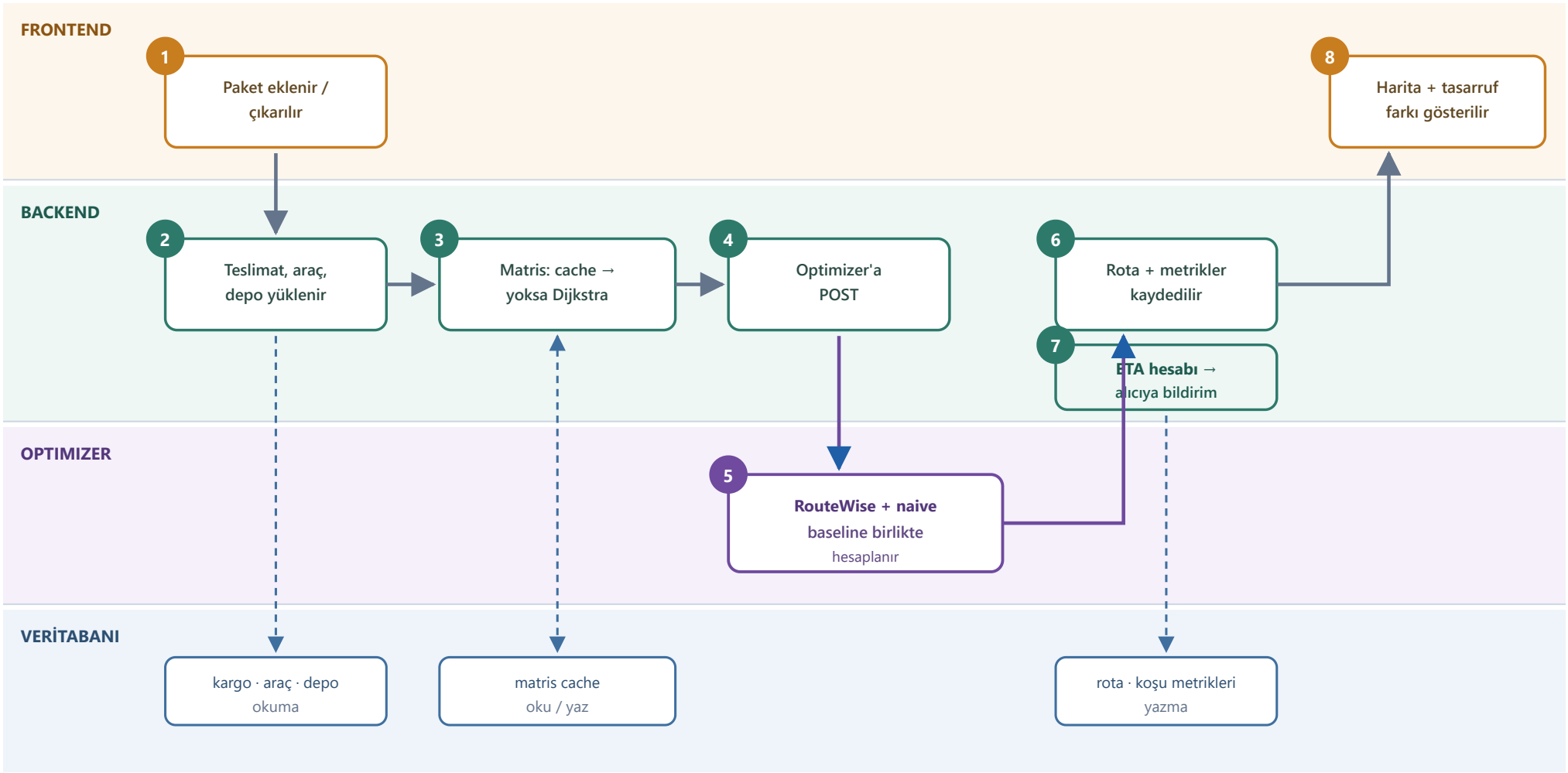
Doğrulama referansı

pgRouting'in **pgr_dijkstra** fonksiyonu, bizim Dijkstra implementasyonumuzun doğruluk referansı:

aynı girdi → aynı mesafe vermeli.

Bu, test stratejisinin zorunlu maddesidir.

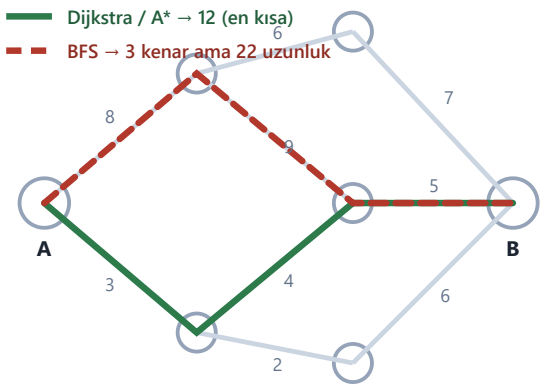
03 Optimizasyon Akışı



ETA(durak i) = depodan çıkış saati + Σ (seyahat süresi + servis süresi) , 1..i arası duraklar için — yol grafındaki segment uzunlukları ve hız sınırlarından

SEVİYE 1 — YOL BULMA

"A noktasından B noktasına hangi yoldan gidilir?" · Bandırma yol grafı üzerinde



Dijkstra

Ağırlıklı grafta en kısa yolu garanti eder

REFERANS OPTİMUM

A*

Dijkstra ile aynı yol, daha az düğüm gezer

haversine sezgiseli

BFS

Kenar ağırlıklarını YOK SAYAR

en az kavşak ≠ en kısa

DFS

Optimallik garantisi yok

keşif sırasına bağlı

Rapor için değerli bulgu

Yol ağı ağırlıklı bir graftır; ağırlıkları yok sayan bir arama algoritması doğru cevabı veremez. BFS'in bu yanlışlığını sayısallaştırmak, algoritma seçimin

Ölçülen metrikler

yol uzunluğu (m) · hesaplama süresi (ms) · gezilen düğüm sayısı · Dijkstra optimumundan sapma (%)

SEVİYE 2 — TUR SIRASI

"Hangi teslimat hangi sırayla, hangi araçla?" · mesafe matrisi üzerinde

Naive (giriş sırası) **RouteWise**



Aynı duraklar, farklı sıra → farklı mesafe, süre ve maliyet

Her koşuda naive baseline de hesaplanır ve kaydedilir: **tasarruf ölçümü bu karşılaştırmadan türetilir.**

Naive — giriş sırası

Taban çizgi: insan dispatcher davranışı. Optimizasyon yok.

Nearest Neighbor

Açgözlü kurucu heuristik. Çok hızlı, kalite orta.

Clarke-Wright savings

Klasik kurucu heuristik. RouteWise'ın 1. adımı.

RouteWise — projenin kendi algoritması

savings (başlangıç) → 2-opt · relocate · exchange · Or-opt (yerel arama) → simulated annealing (yerel optimumdan kaçış)

Her hamlede kapasite ve maksimum rota süresi olurluluğu kontrol edilir.

DFS-exhaustive

yalnızca n ≤ 10

Tüm permütasyonlar →

GERÇEK OPTİMUM

En güçlü nicel iddia DFS-exhaustive küçük örneklerde gerçek optimumu verdiği için → "RouteWise, gerçek optimumun %X içinde" denebilir.

Ölçülen metrikler: toplam mesafe · süre · yakıt (L) · maliyet (TL) · kullanılan araç sayısı · hesaplama süresi (ms) · optimumdan sapma (%)

Maliyet, rota hesaplandıktan SONRA eklenen bir rapor alanı değildir.

Amaç fonksiyonunun parçasıdır — algoritma kararlarını doğrudan bu değerler yönlendirir. "Sadece en kısa yol değil, değişkenler birlikte" cümlesinin kodda karşılığı tam olarak burasıdır.

SERT KISITLAR

Aşılamaz. İhlal eden çözüm geçersizdir — cezalandırılmaz, **reddedilir**.

Araç kapasitesi

Σ paket ağırlığı/hacmi ≤ araç kapasitesi

araç tanımından gelir

Maksimum rota süresi

toplam süre ≤ çalışma günü

"tüm teslimatlar gün içinde bitmeli"

AMAÇ FONKSİYONU BİLEŞENLERİ — minimize edilir

Mesafe maliyeti

toplam mesafe (km)

kaynak: yol grafı + Dijkstra

Yakıt maliyeti

(L/100km × km) × birim fiyat

tüketim araç bazlı

Sürücü maliyeti

süre (saat) × saatlik ücret

servis süresi dahil

Süre maliyeti

toplam seyahat + servis süresi

maxspeed'den hesaplanır

Öncelik ihlali cezası

öncelik × ceza katsayısı

yumuşak kısıt — cezalandırılır

Araç sayısı

sabit maliyet / araç

opsiyonel bileşen

Parametreler nereden gelir

Backend (DB)

yakıt fiyatı · saatlik ücret ·

→ her çağrıda →

Optimizer (istek)

amaç fonksiyonunda kullanılır

Sonuç: yakıt fiyatı veya sürücü maliyeti değiştiğinde kod değişmeden optimizasyon sonuçları değişir.

Neden "sadece en kısa yol" yetmez

İki rota aynı mesafede olabilir ama biri dar sokaklarda yavaş ilerleyip sürücü maliyetini artırır;

bir diğeri daha uzun ama hızlı bir güzergâhtan gidip hem süreyi hem yakıtı düşürür;

bir üçüncüsü yüksek öncelikli paketi güne çok geç bırakır.

Yalnızca mesafeyi minimize eden bir algoritma bu üç farkı göremez. RouteWise'ın amaç fonksiyonu görür.

Pazarlık payı olmayan kural

Sistem tek parça yazılmaz. Takvim sıkışırsa son çalışan katman sunulur. Tek parça geliştirmede proje bitmezse ortada gösterilebilir hiçbir şey kalmaz.

1 KATMAN

Kargo listesi + tek araç + rota (TSP)

- Kargo listesi CRUD: manuel paket ekleme / çıkarma
- Adres → koordinat · yol grafı kurulumu · Dijkstra ile matris
- Tek aracın durak sırası, harita üzerinde çizim

Demo edilebilir çıktı

"Paket ekliyorum, rota haritada yeniden çiziliyor."
Kapasite ve maliyet henüz yok.

2 KATMAN

Çok araç + kapasite kısıtı + maliyet modeli (CVRP)

- Araç filosu · kapasite sert kısıtı
- Clarke-Wright savings → 2-opt / relocate / Or-opt → simulated annealing
- Amaç fonksiyonu: mesafe + yakıt + sürücü + öncelik · naive baseline kaydı

Demo edilebilir çıktı

"3 araçla dağıtım planı ve TL cinsinden maliyet;
naive planla farkı tablo halinde."

3 KATMAN

Maksimum rota süresi + ETA + bildirim

- Çalışma günü süresi sert kısıt olarak algoritmaya girer
- Durak bazında ETA hesabı (maxspeed + servis süresi)
- Alıcıya uygulama içi bildirim + e-posta

Demo edilebilir çıktı

"Paketiniz tahmini 14:20'de teslim edilecek"
bildirimi alıcıya ulaşır.

4 KATMAN

Algoritma karşılaştırma modülü + istatistik raporları

- Seviye 1: Dijkstra · A* · BFS · DFS — yol grafı üzerinde
- Seviye 2: naive · greedy · savings · RouteWise · DFS-exhaustive
- Tablolar, grafikler, haftalık / aylık / yıllık tasarruf analizleri

Projenin en görünür çıktısı

Ölçüm altyapısı Katman 2'den itibaren hazır tutulur —
her koşunun metrikleri baştan kaydedilir.

Neden bu sıra Riskin yoğunlaştığı yer algoritmadır. Optimizer servisi hiçbir altyapıya bağımlı olmadığı için (durumsuz, DB yok) sentetik verilerle tek başına geliştirilip test edilebilir.
Algoritma çalışmazsa proje çalışmaz — bunu CRUD ekranları bittikten sonra keşfetmek istemeyiz.

Bu sayfadaki sayılar ÖRNEKTİR — yalnızca tablo ve grafik yapısını göstermek için konmuştur. Gerçek ölçüm yapılmadı.

Seviye 1 — Yol bulma (tek durak çifti, Bandırma yol grafı)

Algoritma	Yol (m)	Süre (ms)	Düğüm	Sapma
Dijkstra	4 820	12,4	3 140	**0 %**
A*	4 820	6,1	1 020	**0 %**
BFS	7 450	9,8	2 760	**+55 %**
DFS	14 230	4,2	890	**+195 %**

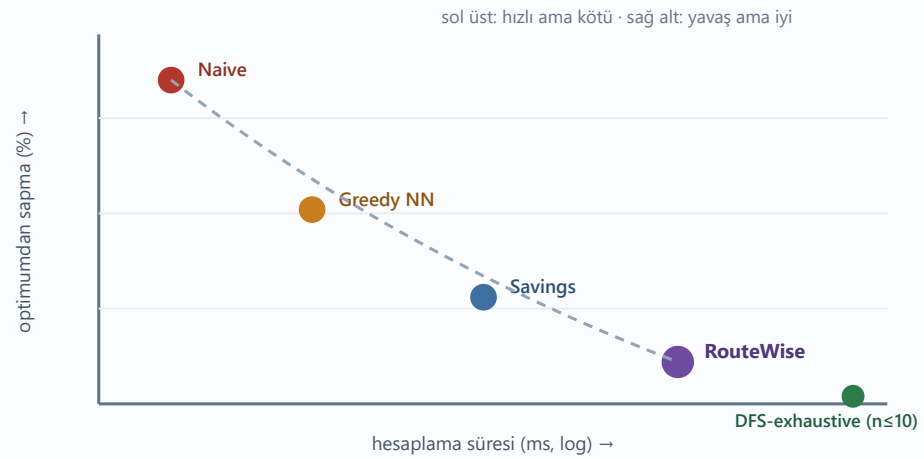
Beklenen bulgu: A* Dijkstra ile aynı yolu daha az düğüm gezerek bulur; BFS ağırlıkları yok saydığı için sapar.

Seviye 2 — Tur sırası (aynı kargo listesi, aynı filo)

Algoritma	km	Yakıt L	TL	Süre ms	Sapma
Naive	142,6	13,1	684	1	**+38 %**
Greedy NN	118,4	10,9	571	8	**+15 %**
Savings	109,2	10,0	527	42	**+6 %**
RouteWise	**103,5**	**9,5**	**498**	1 840	**+2 %**

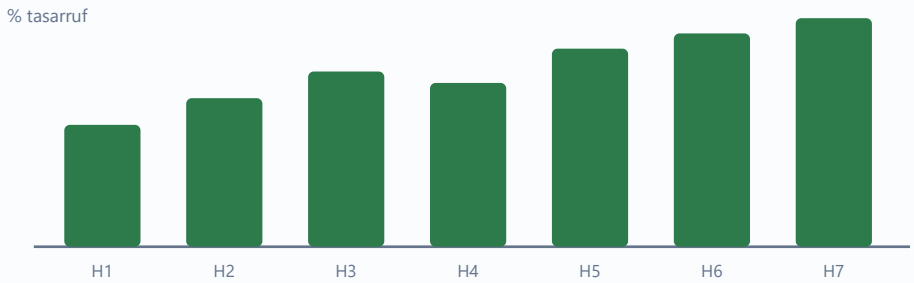
Sapma, küçük örneklerde DFS-exhaustive'in bulduğu gerçek optimuma göre hesaplanır.

Raporun ana grafiği: kalite – hesaplama süresi dengesi



Tasarruf analizleri — zaman serisi

Her optimizasyon koşusunda naive baseline de kaydedildiği için tasarruf yüzdesi geçmişe dönük agrege edilebilir (haftalık / aylık / yıllık).



Ölçülenler: km · yakıt (L) · maliyet (TL) · araç-gün · ortalama teslimat süresi

Sıradaki adım

Mimari tamamlandı. Açık kararlar: adres → koordinat yöntemi · yeniden optimizasyon tetiklemesi · kimlik doğrulama · filo tanımı · Bandırma OSM sınırları · deployment hedefi.
Bu kararlar netleştikten sonra Katman 1 geliştirmesine başlanır.